

Kiste Phase 4 Spec

GitOps Repo + SOPS Secret Management, with Vault-ready secret backend design

One-sentence spec: Kiste Phase 4 turns the multi-repo workspace graph into a GitOps deployment repo, encrypts Kubernetes secrets with SOPS, validates that no plaintext secrets are committed, and designs the secret layer so HashiCorp Vault can be added later without rewriting the system.

1. Phase 4 Position

Phase 4 should focus on GitOps repository generation and secret handling, not on supporting every container registry first. Kiste already understands local repos, remote repos, and multi-repo workspaces. Phase 4 turns that understanding into a deployment source of truth.

Phase	Focus
Phase 1	Local Docker repo scan + token inspection
Phase 2	GitHub/Hugging Face repo scan + CI testing
Phase 3	Multi-repo workspace + system graph + token access matrix
Phase 4	GitOps repo generation + SOPS encrypted secrets + Vault-ready backend interface

2. Main Idea

Kiste should generate a separate GitOps repo that stores Kubernetes desired state. Application repos stay focused on application code. The GitOps repo stores deployment manifests, environment overlays, network policies, config maps, and encrypted secrets.

```
application repos
-> Kiste workspace graph
-> GitOps repo
-> Flux / Argo CD
-> Kubernetes cluster
```

The important security rule is simple: Kiste commits encrypted secrets, never plaintext secrets.

3. GitOps Repo Structure

Recommended output structure for a generated GitOps repo:

```
robot-platform-gitops/
README.md
.sops.yaml

clusters/
dev/
kustomization.yaml
apps.yaml
infrastructure.yaml
prod/
kustomization.yaml
apps.yaml
infrastructure.yaml

apps/
frontend/
deployment.yaml
service.yaml
ingress.yaml
kustomization.yaml
api/
deployment.yaml
```

```

service.yaml
configmap.yaml
kustomization.yaml
worker/
deployment.yaml
kustomization.yaml

infrastructure/
namespaces/
robot-dev.yaml
robot-prod.yaml
network-policies/
api-policy.yaml
redis-policy.yaml

secrets/
api-secret.enc.yaml
redis-secret.enc.yaml
postgres-secret.enc.yaml

```

4. Why SOPS First, Vault Later

SOPS fits GitOps because encrypted YAML files can live safely in Git. Kiste can generate Kubernetes Secret manifests, encrypt them with SOPS, and commit only encrypted files into the GitOps repo.

HashiCorp Vault should remain a later backend because it is more powerful and heavier. Vault is useful when Kiste needs runtime secret lookup, dynamic database credentials, secret rotation, and policy-based access.

Backend	Best use in Kiste
SOPS	Phase 4: Git-friendly encrypted Kubernetes secrets committed to GitOps repo
Vault	Later phase: runtime secret service, dynamic credentials, rotation, policy-based secret access

5. Phase 4 Architecture

```

Kiste CLI
-> Workspace Graph
-> GitOps Generator
-> Secret Backend Interface
-> SOPS Backend
-> Vault Backend later
-> External Secrets Backend later
-> Cloud Secret Manager Backend later
-> GitOps Repo Writer
-> Flux / Argo CD compatible repo

```

The core design choice is to define a general secret backend interface. SOPS is the first implementation, but not the permanent hardcoded answer.

```

class SecretBackend:
def encrypt_secret(self, name: str, data: dict) -> bytes:
...

def decrypt_secret(self, encrypted_file: bytes) -> dict:
...

def validate_config(self) -> bool:
...

```

6. New Commands

GitOps commands

```

kiste gitops init --workspace robot-platform --output ./robot-platform-gitops
kiste gitops generate --env dev
kiste gitops generate --env prod
kiste gitops plan

```

```
kiste gitops validate
```

SOPS secret commands

```
kiste secrets init sops --age
kiste secrets init sops --aws-kms
kiste secrets init sops --gcp-kms
kiste secrets init sops --azure-keyvault

kiste secrets add api --from-env DATABASE_URL --from-env SECRET_KEY --encrypt sops

kiste secrets scan
```

Future Vault command shape

```
kiste secrets init vault --addr https://vault.example.com
kiste secrets backend set vault
kiste secrets sync --backend vault
```

7. Example SOPS Configuration

Kiste should generate a .sops.yaml file. For age-based encryption, the file can look like this:

```
creation_rules:
- path_regex: secrets/*\.enc\.yaml$
age: age1examplepublickeyhere
```

Generated encrypted files should use a clear naming convention such as api-secret.enc.yaml, postgres-secret.enc.yaml, and redis-secret.enc.yaml. Plain files like api-secret.yaml should be rejected from the GitOps repo.

8. Secret Backend Configuration

Kiste should store backend intent in project or workspace config:

```
secrets:
backend: sops
sops:
key_type: age
config_file: .sops.yaml

future_backends:
- vault
- external-secrets
- aws-secrets-manager
- gcp-secret-manager
- azure-key-vault
```

9. GitOps Validation

The gitops validate command should check both structure and security.

```
kiste gitops validate
```

Example validation report:

```
GitOps validation:

[OK] GitOps repo structure valid
[OK] dev cluster folder found
[OK] prod cluster folder found
[OK] Kustomization files valid
[OK] API Deployment generated
[OK] Worker Deployment generated
[OK] NetworkPolicy generated
[OK] SOPS config found
[OK] Encrypted secret files found
[FAIL] Plaintext secret detected: secrets/api-secret.yaml
[WARN] prod ingress host missing
```

10. Plaintext Secret Detection

Kiste should scan the GitOps repo before commit and fail if plaintext secrets are found.

```
kiste secrets scan
```

Minimum checks:

- Reject files named secret.yaml, *-secret.yaml, or .env unless encrypted.
- Reject Kubernetes Secret manifests outside approved encrypted paths.
- Warn on suspicious keys such as PASSWORD, TOKEN, SECRET, DATABASE_URL, REDIS_URL, AWS_ACCESS_KEY_ID, and PRIVATE_KEY.
- Allow only *.enc.yaml files in the secrets/ directory by default.
- Print exact file path and reason for every violation.

11. What Phase 4 Should Generate

- Kubernetes Deployment, Service, ConfigMap, and NetworkPolicy files based on the Phase 3 workspace graph.
- Kustomize overlays for dev and prod environments.
- Cluster folder structure compatible with Flux or Argo CD workflows.
- SOPS configuration for encrypted secrets.
- Encrypted Kubernetes Secret manifests.
- Validation reports for GitOps structure and plaintext secret detection.
- A secret backend interface that can support SOPS now and Vault later.

12. What Phase 4 Should Not Do Yet

- No full Vault integration yet.
- No dynamic database credentials yet.
- No automatic production cluster deployment.
- No automatic secret rotation.
- No multi-cloud secret synchronization.
- No complex policy engine.

Phase 4 is mainly: workspace graph -> GitOps repo -> encrypted secrets with SOPS.

13. Acceptance Criteria

Phase 4 is complete when this workflow works:

```
kiste workspace scan
kiste workspace graph
kiste gitops init --workspace robot-platform --output ./robot-platform-gitops
kiste secrets init sops --age
kiste gitops generate --env dev
kiste secrets add api --from-env DATABASE_URL --from-env SECRET_KEY --encrypt sops
kiste secrets scan
kiste gitops validate
```

And Kiste produces:

- GitOps repo structure.
- Kubernetes manifests from the workspace graph.

- Customize folders for dev and prod.
- SOPS configuration.
- Encrypted Kubernetes Secret files.
- Plaintext secret detection.
- GitOps validation report.
- Vault-compatible secret backend interface for later.

14. References

Useful implementation references:

- OpenGitOps principles: <https://opengitops.dev/>
- SOPS documentation: <https://getsops.io/docs/>
- Flux SOPS guide: <https://fluxcd.io/flux/guides/mozilla-sops/>
- HashiCorp Vault documentation: <https://developer.hashicorp.com/vault>